

Assignment 21.3 Ai Assisted Coding

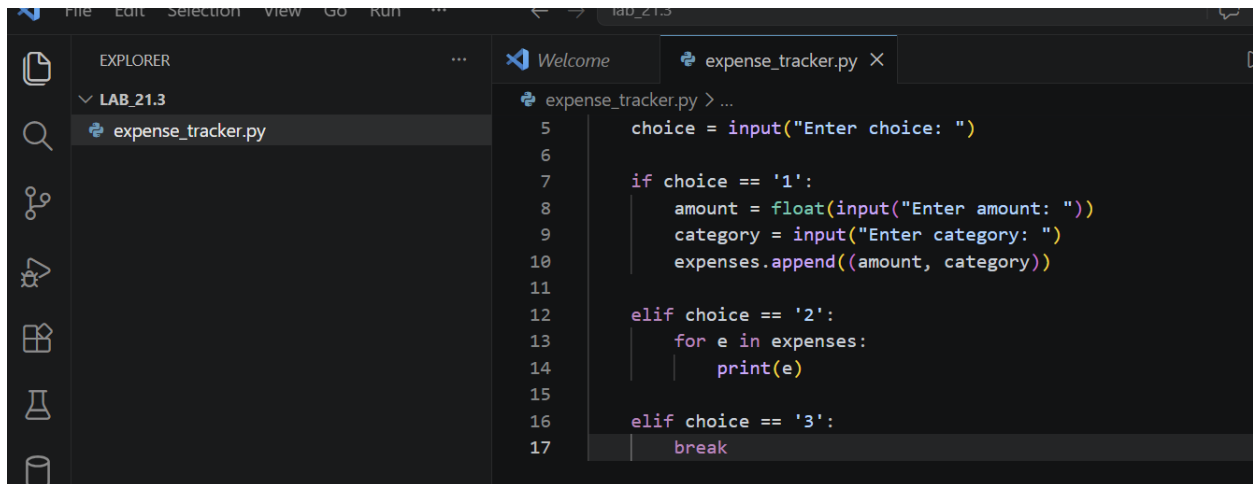
Htno:2303a51288

Btno:05

Task 1: AI-Assisted Expense Tracker Application using Python

Prompt: Generate a basic Python program to record daily expenses with amount and category. Then improve it by adding monthly summary, input validation, and modular functions

Code:



```
5 choice = input("Enter choice: ")
6
7 if choice == '1':
8     amount = float(input("Enter amount: "))
9     category = input("Enter category: ")
10    expenses.append((amount, category))
11
12 elif choice == '2':
13     for e in expenses:
14         print(e)
15
16 elif choice == '3':
17     break
```

Output:



```
PS C:\Users\uppus\Downloads\lab_21.3> python expense_tracker.py
2. View Expenses
3. Exit
Enter choice: 1
Enter amount: 200
Enter category: food

1. Add Expense
2. View Expenses
3. Exit
Enter choice: 2
(200.0, 'food')

3. Exit
```

- The program stores expenses in a list
- User can:
- Add expense (amount + category)
- View expenses

Corrtected Code:

```
1 expenses = []
2
3 def add_expense():
4     try:
5         amount = float(input("Enter amount: "))
6         category = input("Enter category: ")
7         expenses.append((amount, category))
8         print("Expense added successfully!")
9     except:
10        print("Invalid input!")
11
12 def view_expenses():
13     if len(expenses) == 0:
14         print("No expenses recorded!")
15     else:
16         print("\nExpenses List:")
17         for amount, category in expenses:
18             print("Category:", category, "| Amount:", amount)
19
20 while True:
21     print("\n1. Add Expense")
22     print("2. View Expenses")
23     print("3. Exit")
24
25     choice = input("Enter choice: ")
26
27     if choice == '1':
28         add_expense()
29     elif choice == '2':
```

Output:

```
PS C:\Users\uppus\Downloads\lab_21.3> python expense_tracker.py
1. Add Expense
2. View Expenses
3. Exit
Enter choice: 1
Enter amount: 100
Enter category: food
Expense added successfully!

1. Add Expense
2. View Expenses
3. Exit
Enter choice: 2

Expenses List:
Category: food | Amount: 100.0
```

Explanation:

- Code is divided into functions:
- `add_expense()` → adds expense
- `view_expenses()` → displays expenses
- Added **input validation** using **try-except**
- Displays **user-friendly output format**
- Shows message when no data is available

Task 2: Version Control Integration using Git – Student Grade Calculator

Prompt: Create a Python program to calculate average marks. Then enhance it using AI by adding grade classification and percentage calculation, and manage the project using Git with branching and merging."

Code: Initial Code

```
task2.py > ...  
1  marks = [80, 75, 90]  
2  
3  avg = sum(marks) / len(marks)  
4  print("Average:", avg)
```

```
PS C:\Users\uppus\Downloads\lab_21.3> python task2.py  
PS C:\Users\uppus\Downloads\lab_21.3> python task2.py  
Average: 81.66666666666667  
PS C:\Users\uppus\Downloads\lab_21.3> 
```

Output:

Explanation:

- Stores marks in a list
- Calculates average using `sum()` and `len()`
- Displays only average

Corrected Code:

```

1  marks = []
2
3  n = int(input("Enter number of subjects: "))
4
5  for i in range(n):
6      m = int(input("Enter marks: "))
7      marks.append(m)
8
9  avg = sum(marks) / len(marks)
10 percentage = avg
11
12 if percentage >= 90:
13     grade = 'A'
14 elif percentage >= 75:
15     grade = 'B'
16 elif percentage >= 50:
17     grade = 'C'
18 else:
19     grade = 'Fail'
20
21 print("Average:", avg)
22 print("Percentage:", percentage)
23 print("Grade:", grade)

```

Output:

```

● PS C:\Users\uppus\Downloads\lab_21.3> python task2.py
Enter number of subjects: 2
Enter marks: 60
Enter marks: 95
Average: 77.5
Percentage: 77.5
Grade: B
○ PS C:\Users\uppus\Downloads\lab_21.3>

```

Explanation:

- Takes **user input** for marks
- Calculates:
 - Average
 - Percentage
- Assigns grade based on percentage
- Uses conditional statements for classification
- More realistic and user-friendly

Task 3: Password Validation Module with AI-Generated Commit Messages

Prompt: Create a Python program to validate a password with rules like minimum length and special character requirement. Also generate meaningful commit messages for the updates.

Code: Initial Code

```
1 password = input("Enter password: ")
2
3 if password == "1234":
4     print("Valid")
5 else:
6     print("Invalid")
```

Output:

```
● PS C:\Users\uppus\Downloads\lab_21.3> python task3.py
Enter password: 1234
Valid
● PS C:\Users\uppus\Downloads\lab_21.3> python task3.py
Enter password: sony@0610
Invalid
○ PS C:\Users\uppus\Downloads\lab_21.3> █
```

Explanation:

- Takes password input from user
- Checks only one fixed password ("1234")
- Prints valid or invalid

Corrected Code:

```
task3.py > ...
1 import re
2
3 password = input("Enter password: ")
4
5 if len(password) < 6:
6     print("Password too short!")
7 elif not re.search("[@#$$%^&*]", password):
8     print("Password must contain special character!")
9 else:
10    print("Valid password")
```

Output:

```
PS C:\Users\uppus\Downloads\lab_21.3> python task3.py
Enter password: Sony@1234
Valid password
PS C:\Users\uppus\Downloads\lab_21.3> 1234
1234
PS C:\Users\uppus\Downloads\lab_21.3> 
```

```
PS C:\Users\uppus\Downloads\lab_21.3> git config --global --list
PS C:\Users\uppus\Downloads\lab_21.3> git add .
● >> git commit -m "Added password validation with minimum length and special character requirement"
[master (root-commit) c01d543] Added password validation with minimum length and special character requirement
3 files changed, 68 insertions(+)
create mode 100644 expense_tracker.py
create mode 100644 task2.py
create mode 100644 task3.py
```

Explanation:

- Uses **regular expressions (re module)**
- Checks:
- Minimum length (6 characters)
- Special character requirement
- More secure and realistic
- Provides meaningful messages

Task 4: Quiz Management System using Pair Programming with AI Assistance and Git Integration

Prompt: Create a simple quiz program with questions and answers. Then enhance it using AI by adding score calculation and input validation. Simulate collaboration using Git branching and merging.

Code: Initial Code

```

task4.py > ...
1  questions = {
2      "Capital of India?": "Delhi",
3      "2+2?": "4"
4  }
5
6  for q in questions:
7      ans = input(q + " ")
8      if ans == questions[q]:
9          print("Correct")
10     else:
11         print("Wrong")

```

Output:

```

PS C:\Users\uppus\Downloads\lab_21.3> python task4.py
Capital of India? Delhi
Correct
2+2? 4
Correct
PS C:\Users\uppus\Downloads\lab_21.3>

```

Explanation:

- stores questions and answers using a dictionary
- Loops through each question
- Takes user input
- Checks correctness and prints result

Corrected Code:

```
1  questions = {
2      "Capital of India?": "Delhi",
3      "2+2?": "4"
4  }
5
6  score = 0
7
8  for q in questions:
9      ans = input(q + " ")
10
11     if ans.lower() == questions[q].lower():
12         print("Correct")
13         score += 1
14     else:
15         print("Wrong")
16
17 print("Final Score:", score)
```

Output:

```
TERMINAL powershell + v
● PS C:\Users\uppus\Downloads\lab_21.3> python task4.py
Capital of India? Delhi
Correct
2+2? 4
Correct
Final Score: 2
❖ PS C:\Users\uppus\Downloads\lab_21.3> 
```



```
PS C:\Users\uppus\Downloads\lab_21.3> git checkout master
>> git merge feature-score
Fast-forward
 task4.py | 6 -----
 1 file changed, 6 deletions(-)
○ PS C:\Users\uppus\Downloads\lab_21.3> git log
commit a28985f0c93f89d5e6f3f52170c82e85aaf4cb31 (HEAD -> master, feature
-score)
Author: Snehanjali Uppu <snehanjali@gmail.com>
Date: Thu Apr 2 10:38:17 2026 +0530

    Added score calculation and input validation to quiz system

commit 7d07ea86b3b2e9e2e02c178df73cbcdfbfc2e5d2
Author: Snehanjali Uppu <snehanjali@gmail.com>
Date: Thu Apr 2 10:31:04 2026 +0530

    Initial commit: basic quiz program
```

Explanation:

Basic quiz program was created (questions and answers)

Score calculation was added using AI assistance

Git was used to merge changes from branch to main